

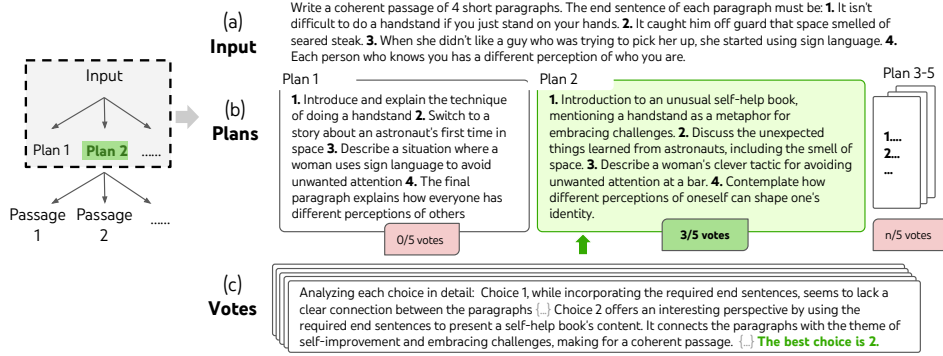


Figure 4: A step of deliberate search in a randomly picked Creative Writing task. Given the input, the LM samples 5 different plans, then votes 5 times to decide which plan is best. The majority choice is used to consequently write the output passage with the same sample-vote procedure.

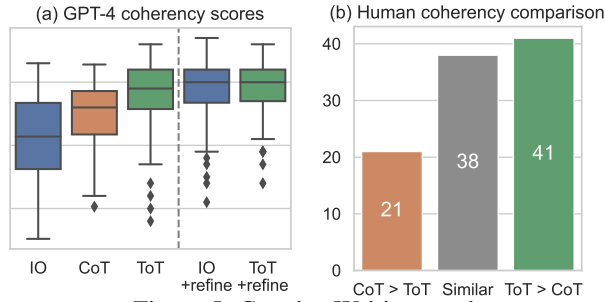


Figure 5: Creative Writing results.

Method	Success Rate (%)		
	Letter	Word	Game
IO	38.7	14	0
CoT	40.6	15.6	1
ToT (ours)	78	60	20
+best state	82.4	67.5	35
-prune	65.4	41.5	5
-backtrack	54.6	20	5

Table 3: Mini Crosswords results.

it improves IO coherency score from 6.19 to 7.67, and ToT coherency score from 7.56 to 7.91. We believe it could be thought of as a third approach to thought generation in the ToT framework, where new thoughts can arise from refining old thoughts instead of i.i.d. or sequentially generated.

4.3 Mini crosswords

In Game of 24 and Creative Writing, ToT is relatively shallow — at most 3 thought steps are needed to reach the final output. Here we explore 5×5 mini crosswords as a harder search problem involving natural language. Again, the goal is not just to solve the task, as more general crosswords can be readily solved with specialized NLP pipelines [34] that leverages large-scale retrieval instead of LM. Rather, we aim to explore the limit of LM as a general problem solver that explores its own thoughts and guides its own exploration with deliberate reasoning as heuristics.

Task setup. We scrape data from GooBix, which contains 156 games of 5×5 mini crosswords. As we observe adjacent games contain similar clues, we use 20 games with indices 1, 6, ..., 91, 96 for testing, and games 136, 141, 146, 151, 156 for prompting. For each task, the input describes the 5 horizontal clues and 5 vertical clues, and the output should be a board of $5 \times 5 = 25$ letters to solve the crosswords. For evaluation, we consider three levels of success: the portion of correct letters (25 per game), words (10 per game), and games.

Baselines. We provide 5 example input-output pairs in the IO prompt, and in the CoT prompt additionally include intermediate words in the order h1..5 then v1..5. We run each prompt for 10 samples and average the results.

ToT setup. We leverage a depth-first search (Algorithm 2) that keeps exploring the most promising subsequent word clue until the state is no longer promising, then backtrack to the parent state to explore alternative thoughts. To make search tractable, subsequent thoughts are constrained not to change any filled words or letters, so that the ToT has at most 10 intermediate steps. For thought generation, at each state we translate all existing thoughts (e.g. “h2.motor; h1.tasks” for the state in Figure 6(a)) into letter constraints for remaining clues (e.g. “v1.To heap: tm____;...””) and prompt a proposal prompt 5 times to come up with candidates for where and what to fill in the next word. Importantly, we also prompt the LM to give a confidence level for different thoughts, and aggregate

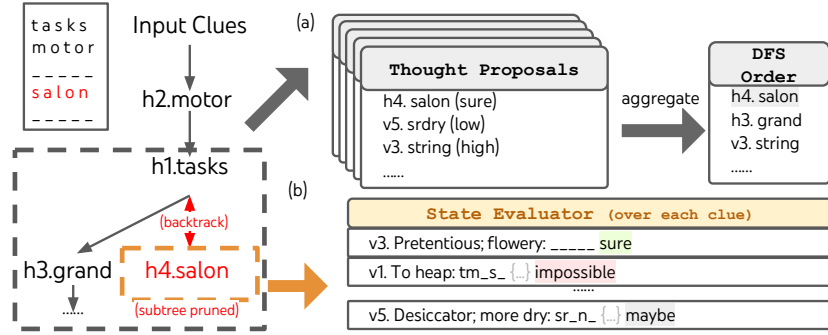


Figure 6: In Mini Crosswords, (a) how thoughts are proposed and aggregated in a priority queue for depth-first search (DFS), and (b) how a state is evaluated based on the possibility of filling in each remaining word clue, and pruned if any remaining clue is deemed not possible to fill by the LM. Then DFS backtracks to the parent state and explore the next promising thought for clue.

these across proposals to obtain a sorted list of next thoughts to explore (Figure 6(a)). For state evaluations, we similarly translate each state into letter constraints for remaining clues, then evaluate for each clue if it is possible to fill given the constraints. If any remaining clue is deemed “impossible” to fill in (e.g. “v1. To heap: tm_s_”), then the exploration of the state’s subtree is pruned and DFS backtracks to its parent to explore the next promising thought. We limit DFS search steps to 100, and simply render the deepest explored state (the first explored one if multiple) into the final output.

Results. As shown in Table 3, IO and CoT prompting methods perform poorly with a word-level success rate less than 16%, while ToT significantly improves all metrics, achieving a word-level success rate of 60% and solving 4 out of 20 games. Such an improvement is not surprising, given IO and CoT lack mechanisms to try different clues, make changes to decisions, or backtrack.

Oracle and ablation studies. When outputting from the oracle best DFS state (instead of the heuristically determined best state) per task, ToT performance is even higher and actually solves 7/20 games (Table 3, “+best state”), indicating our simple output heuristics can be readily improved. Interestingly, sometimes when the crosswords game is actually solved, the state evaluator might still deem some words as “impossible” and prune — possibly because 5×5 crosswords by design have some rare or obsolete words that GPT-4 cannot recognize². Given the state evaluation as a pruning heuristic is imperfect, we also explore ablating the pruning, and find the performance generally worse (Table 3, “-prune”). However, it could actually find the correct solution for 4/20 games (though only outputting 1 via heuristic), 3 of which are games ToT+pruning cannot solve within 100 steps. Thus, better heuristics for DFS pruning are critical for problem solving in this case. Lastly, we confirm the importance of backtracking by running an ablation that keeps filling the most promising clue for at most 20 steps, allowing overwrites. This is similar to a “greedy” BFS search with breadth limit of $b = 1$, and performs poorly with a word level success of only 20% (Table 3, “-backtrack”).

5 Related Work

Planning and decision making. Smart planning and decision making are critical to achieving predefined goals. As they are trained on vast amount of world knowledge and human examples, LMs are known to have already absorbed rich commonsense that makes it possible to propose reasonable plans conditioned on problem setting and environmental states [12, 42, 37, 13, 35, 41, 40]. Our proposed ToT approach extends existing planning formulations by considering multiple potentially feasible plans simultaneously at each problem-solving step, and proceeding with the most promising ones. The integration between thought sampling and value feedback organically integrates planning and decision-making mechanisms, enabling effective search inside a solution tree. On the other hand, traditional decision-making procedures usually require training dedicated reward and policy models as in reinforcement learning (for example CHAI [33]), whereas we use the LM itself to provide the value estimates for decision making. RAP [9] is a concurrent work that treats language model

²For example, “agend” is an obsolete form of “agendum”, but GPT-4 deems it a typo for “agenda”. External retrieval or web interaction could augment LM for problem solving under knowledge uncertainty.

reasoning as planning with its internal world model, and proposes a MCTS-based method similar to ToT. However, its tasks are simpler than ours, and its framework lacks the modularity to incorporate different tree search algorithms.

Self-reflection. Using LLMs to assess the viability of their own predictions is becoming an increasingly important procedure in problem solving. [28, 20, 24] introduced the “self-reflection” mechanism, in which LMs provide feedback to their generation candidates. [4] improves LMs code generation accuracy by injecting feedback messages generated by the LM itself based on its code execution results. Similarly, [17] also introduces “critic” or review steps over the actions and states, deciding the next action to take in solving computer operation tasks. Another recent work very relevant to ours is “self-eval guided decoding” [39]. Similar to our method, self-eval decoding also follows a tree-search procedure with leaves sampled from stochastic beam search decoding, which are then evaluated by LLM itself with carefully prepared self-eval prompts. Their approach however, uses the PAL formulation [8] which represents thoughts as codes, which makes it difficult to tackle challenging tasks like creative writing which we consider in this paper. Our Tree-of-Thought formulation is thus more versatile and handles challenging tasks on which GPT-4 only achieves very low accuracy with standard prompts.

Program-guided LLM generation. Our proposal is also related to recent advancements that organize LM’s behavior with systematic procedures [14, 44, 6, 43] or symbolic program guidance. For example, Schlag et al. [27] embeds LMs in an algorithmic search procedure to help solve problems like question answering step-by-step, in which the search trees are expanded by relevant paragraphs that might provide answers. This approach however differs from ours in that trees are expanded by sampling external paragraphs instead of the LM’s own thoughts, and there is no reflection or voting steps. Another approach, LLM+P [18], goes one step further and delegates the actual planning process to a classical planner.

Classical search methods. Last but not least, our approach can be treated as a modern rendition of classical search methods for problem solving. For example it can be considered as a heuristic search algorithm like A^* [10], in which the heuristic at each search node is provided by the LM’s self-assessment. From this perspective, our method is also related to NeuroLogic A^* esque decoding [19], which is inspired by A^* search but introduces look-ahead heuristics that are efficient for LMs to improve the beam-search or top-k sampling decoding. This method however is constrained to sentence generation tasks, whereas our framework are designed for complex, multi-step problem solving guarded by value feedback.

6 Discussion

Limitations and future directions. Deliberate search such as ToT might not be necessary for many existing tasks that GPT-4 already excels at (see Appendix B.1), and as an initial step this work only explores three relatively simple tasks that challenges GPT-4 (see Appendix B.2 for some GPT-3.5 experiment results) and calls of better search and planning abilities incorporated with LMs. However, as we begin to deploy LMs for more real-world decision making applications (e.g. coding, data analysis, robotics, etc.), more complex tasks could emerge and present new opportunities to study these research questions. Also, search methods like ToT requires more resources (e.g. GPT-4 API cost) than sampling methods in order to improve task performances, but the modular flexibility of ToT allows users to customize such performance-cost tradeoffs, and ongoing open-source efforts [32] should readily reduce such costs in the near future. More details about cost and efficiency are in Appendix B.3. Lastly, this work focuses on using an off-the-shelf LM, and fine-tuning LMs using a ToT-style high-level counterfactual decision making (e.g. deliberating over potential choices for the next paragraph, instead of predicting the next token) might present opportunities to enhance the problem-solving capabilities of LMs.

Conclusion. The associative “System 1” of LMs can be beneficially augmented by a “System 2” based on searching a tree of possible paths to the solution to a problem. The Tree of Thoughts framework provides a way to translate classical insights about problem-solving into actionable methods for contemporary LMs. At the same time, LMs address a weakness of these classical methods, providing a way to solve complex problems that are not easily formalized, such as creative writing. We see this intersection of LMs with classical approaches to AI as an exciting direction.